DOCUMENT INTERNE · NE PAS DIFFUSER DE SECRETS

Hubera — Récapitulatif technique 20 septembre 2026

Proxies YouTube MHC, bibliothèque web, Fuel, HuberaPress, Jobs, Git

Hubera Cloud / Pavel Delhomme · VPS Contabo `vmi1296373` · date de session 20 septembre 2026

Classification : technique ops + code

Périmètre : holding Hubera (pas Cloudity)

Format : A4, impression @page

1. Couverture et sommaire

Ce document consigne le travail réellement livré le 20 septembre 2026 : rotation de proxies HTTP pour Hubera Music (MHC / YTMusic), steward Python hors-processus, bibliothèque web fenêtrée 1.3.265, jauges Fuel 1.4.142, chrome HuberaPress, titres Hubera Jobs, renames GitHub, et règles d'exploitation Portainer. Les mots de passe, jetons OAuth, cookies Innertube et fichiers `.env` sont exclus. Les identifiants de connexion cités se limitent à l'e-mail staff connu.

Le chapitre 4 (système proxy YouTube) est le cœur du livrable : il décrit le diagnostic (IP datacenter Contabo, `LOGIN_REQUIRED`, HTTP 429, pool global partagé), l'inventaire des sources, le modèle `ProxyEntry`, l'algorithme de lease exclusif par `userId`, la course `Promise.any` à trois proxies pour résoudre le même titre, le binding googlevideo (nsig), le relais Node `fetchUrlViaProxy`, le gate yt-dlp, le scan bibliothèque par compte, le steward, les stats, et ce que l'on refuse volontairement (OAuth via proxy gratuit, SOCKS côté Node).

1. Couverture et sommaire (cette page)
2. Périmètre et règles d'exploitation
3. Cartographie des dépôts GitHub, SHA poussés, dossiers locaux
4. Système proxy YouTube (chapitre principal)
    1. 4.1 Diagnostic : une IP, N comptes, un titre
    2. 4.2 Sources d'harvest et cycle de vie du pool
    3. 4.3 Modèle `ProxyEntry` et constantes
    4. 4.4 Lease exclusif `leasePoolForUser`
    5. 4.5 Sélection : `nextYoutubeProxy`, attempts, stripe
    6. 4.6 Course de résolution yt-dlp `-g` et `boundProxy`
    7. 4.7 Relais CONNECT/TLS et anti-SSRF
    8. 4.8 `ytDlpGate` : cooldown user vs global
    9. 4.9 Scan `libraryHealth` et `libraryWarmSweep`
    10. 4.10 Steward Python et test réel 38/28 s
    11. 4.11 Fichiers, stats, hors-périmètre, live 1.3.265
5. Bibliothèque web MHC 1.3.265
6. Hubera Fuel 1.4.142
7. HuberaPress (admin Next vs Django staff)
8. Hubera Jobs — titres live
9. `apps/` vs `products/`
10. Déploiements production du 20 septembre
11. Exploitation quotidienne

2. Limites et suite

3. Annexe : versions, applicationId, glossaire

> Lecture conseillée pour un opérateur de nuit : §2 (règles), §4.4–4.8 (lease + boundProxy + gate), §11 (429), tableau des SHA §3. Ne pas lancer `docker compose down -v`. Ne pas merger Music `fix/stream-live-warm-247` sur une branche GitHub « prod » sans décision explicite : le live est overlay image, pas forcément le SHA `main` distant.

2. Périmètre et règles

Hubera (`hubera.cloud`) est le holding : catalogue, vitrine `apps/www`, paquets `@hubera/*`, scripts NPM/Portainer, liens vers les clones Perso. **Cloudity** est une archive (ancien monorepo suite). Ce récap n'ajoute aucun produit Cloudity au catalogue. Les domaines historiques `*.delhomme.ovh` et `jobbingtrack.com` restent en production jusqu'au dual-alias fumé ; on n'édite pas les hosts Nginx Proxy Manager existants, on n'en crée que si un sous-domaine Hubera manque.

VPS : Contabo `vmi1296373`, IPv4 publique `95.111.227.204`, SSH alias `pavel-server`, reverse-proxy `nginx-proxy-manager_npm_1`, réseau Docker partagé `shared-network-copy`. Les stacks Portainer sont une app = un Git = une stack. Les volumes utilisateurs sont immuables au sens ops : `ytmusic_ytmusic_data` (SQLite MHC, caches, cookies machine, steward `live.json`) et `gasoil_api_data` (Fuel). Un `down -v` les détruit. Interdit, y compris « pour tester un rebuild propre ».

2.1 Overlay Android `-r`

Les APK de production se mettent à jour par overlay `adb install -r` (même signature, même `applicationId`). Désinstaller pour « réinstaller propre » casse les identifiants Play/stores internes, les bases locales et le QR login. Les flavors lab (`.dev`, etc.) ne sont pas la cible des overlays prod. Les trois identifiants immuables de ce jour :

Produit	applicationId Android	Conséquence
Hubera Music (MHC / YTMusic)	`ovh.delhomme.ytmusic`	Pas de rename package, pas de flavor prod distincte
Hubera Fuel (GasoilTracking)	`com.gasoiltracking.app`	1.4.142 code ; overlay APK pas encore posé sur tous les tél. (voir §6 et §12)
Hubera Jobs (JobbingTrack mobile)	`com.example.jobbingtrack_mobile`	Identifiant historique ; branding UI seulement

2.2 Identité holding vs runtime

Le manifeste / canal OTA peut dire `cloud.hubera.music` ou `cloud.hubera.cms` : ce n'est pas l'`applicationId`. GitHub peut s'appeler `HuberaFuel` alors que le dossier local reste `GasoilTracking`. Le compose Portainer continue d'utiliser les noms de services historiques (`ytmusic`, `gasoil-tracking-web`, `jobbingtrack-prod-frontend`) pour ne pas casser NPM.

> Règles non négociables du 20 sept. : (1) holding = Hubera, pas Cloudity ; (2) overlay `-r` uniquement ; (3) jamais `down -v` ; (4) `applicationId` listés ci-dessus immuables ; (5) pas de pull GHCR Music « au cas où » si l'image locale overlay suffit ; (6) pas de cookies OAuth utilisateur dans un proxy public.

2.3 Pourquoi ces règles existent (contexte MHC)

MHC résout des URLs googlevideo via yt-dlp et Innertube. L'IP sortante du VPS est une IP de datacenter Contabo, déjà saturée par YouTube (botcheck, `LOGIN_REQUIRED`, 429). Le volume Docker contient l'état de santé des titres, les cookies machine (pas les OAuth user pour le chemin proxy gratuit), et le fichier steward. Recréer le volume = perdre des jours de scan `track_health` et le pool chaud. Changer l'`applicationId` = deux apps côte à côte sur le téléphone, QR cassé, stores divergents.

3. Cartographie GitHub après rename et commits du 20 septembre

Les remotes `origin` ont été retargetés vers les nouveaux noms GitHub. Les dossiers locaux sous `~/Documents/Dev/Perso/` n'ont **pas** été renommés (évite de casser workspace Cursor, symlinks Hubera `products/`, scripts, et habitudes). Les SHA ci-dessous sont ceux poussés le 20 septembre 2026, vérifiés par `git rev-parse` et `git log -1` en fin de session.

Dossier local (inchangé)	SHA court	Branche	Remote GitHub	Sujet du commit
`YTMusic`	`117773c`	`fix/stream-live-warm-247`	`PavelDelhomme/YTMusic`	pools proxy par compte MHC, steward, biblio web fenêtrée
`GasoilTracking`	`59005f6`	`prod`	`PavelDelhomme/HuberaFuel`	historique des jauges par véhicule (1.4.142)
`VTCBuilder`	`306b712`	`feature/blocks-implementation-tests`	`PavelDelhomme/VTCBuilder`	chrome admin HuberaPress et Django sous `/django-admin/`
`JobbingTrack`	`6cc8ba15`	`dev`	`PavelDelhomme/HuberaJobs`	titres Hubera Jobs sur le web, login et compose
`HuberaWordPress`	`aa4e07a`	`main`	`PavelDelhomme/HuberaPress`	`/admin/` vers Next HuberaPress, Django en `/django-admin/`
`Hubera`	`64a9876`	`main`	`PavelDelhomme/Hubera`	catalog HuberaPress / HuberaFuel / HuberaJobs et apps vs products

3.1 Renames GitHub uniquement

Ancien nom GitHub	Nouveau nom GitHub	Dossier local	URL origin (fetch)
HuberaWordPress	HuberaPress	`HuberaWordPress`	`git@github.com:PavelDelhomme/HuberaPress.git`
GasoilTracking	HuberaFuel	`GasoilTracking`	`git@github.com:PavelDelhomme/HuberaFuel.git`
JobbingTrack	HuberaJobs	`JobbingTrack`	`git@github.com:PavelDelhomme/HuberaJobs.git`

YTMusic, VTCBuilder et Hubera n'ont pas changé de nom de remote. Les clones extraits la veille (ID, Mail, Drive, Pass, Calendar, Contacts, Notes, Photos, Office) restent hors de ce récap sauf mention catalogue.

3.2 Conséquence workspace

Le workspace Cursor ouvre toujours `products/music` → clone `YTMusic`, `products/fuel` → `GasoilTracking`, `products/jobs` → `JobbingTrack`, `products/cms` → `HuberaWordPress`. Un `git clone` neuf depuis GitHub donnerait un dossier `HuberaFuel` : ce n'est pas l'arbre actuel. Scripts et README meta doivent continuer à citer le chemin local historique.

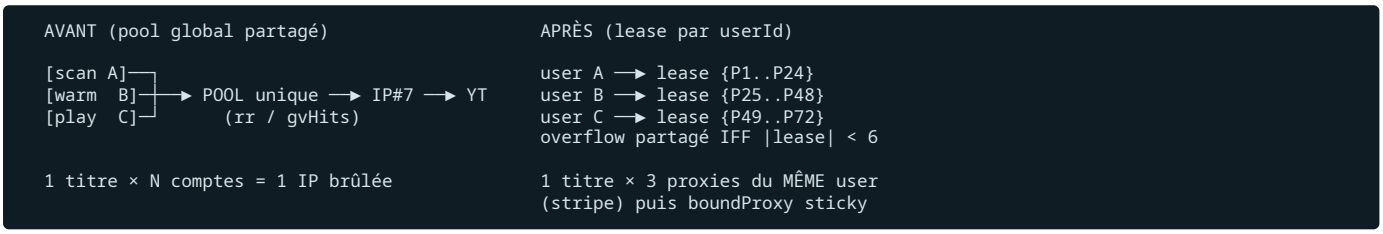
> Music n'est pas mergée sur une branche GitHub nommée `prod`. Le live Portainer est une image overlay `ytmusic:1.3.265-lib` construite depuis `fix/stream-live-warm-247` @ `117773c`. Ne pas inférer « GitHub main = prod Music ».

Commit Hubera `64a9876` : `catalog.json`, `apps/README.md`, `products/README.md`, lien `products/www`, workspace. Commit Fuel `59005f6` : ~795 insertions (table SQLite, snapshot schema 2, UI timeline, tests). Commit Jobs `6cc8ba15` : 12 fichiers, remplacement de libellés JobbingTrack → Hubera Jobs (layout, login, register, maintenance, pageTitles, compose). Commit Press `aa4e07a` + VTCBuilder `306b712` : séparation des chemins admin.

4. Système proxy YouTube — diagnostic

Hubera Music tourne sur le VPS Contabo. YouTube classe cette IPv4 comme datacenter. Les symptômes observés avant le travail du 20 septembre : réponses Innertube / yt-dlp `LOGIN_REQUIRED` (« Sign in to confirm you're not a bot »), HTTP 429 trop de requêtes, 5xx googlevideo (502/503/504/520–523), et parfois un titre qui « marche une fois » puis 403 sur les ranges suivants.

Le défaut architectural d'avant le lease : **un pool global unique**, round-robin entre tous les comptes MHC. Conséquence : si le scan `libraryHealth` et le warm de l'utilisateur A, plus l'écoute live de B, plus le prefetch de C, tapent le même proxy (ou la même poignée de proxies « encore vivants »), YouTube voit *une* IP sortante saturée pour *N* identités applicatives. Un titre demandé simultanément par le health-check et par le player consomme le même budget 429. Le pool « intelligent » (score gvHits) empire le phénomène : tout le monde se rue sur le même gagnant googlevideo.



Le correctif n'est pas « plus de proxies gratuits » tout seul : sans isolation, 600 endpoints se réduisent à 5 gagnants gvHits et on rejoue le même incident. Le correctif est : (1) harvest large + steward pré-filtré ; (2) lease exclusif hashé par `userId` ; (3) course de 3 proxies *du pool user* pour *un* `videoId` ; (4) coller ensuite tout le googlevideo (ranges) à l'IP qui a décrypté le nsig ; (5) cooldown yt-dlp *par user*, global seulement si 3 users distincts bot/429 en 2 minutes ; (6) scan et warm qui passent le `userId` réel, plus un identifiant YouTube orphelin.

4.1 Ce que YouTube lie à l'IP

yt-dlp `-g` (get-url) produit une URL `*.googlevideo.com` contenant un paramètre de signature / nsig calculé dans le player JS. Ce nsig est validé côté Google en association avec l'IP (et souvent l'UA) qui a demandé le player. Si Node, ensuite, fait un `GET` Range `bytes=0-1` ou le corps audio via une *autre* IP (autre proxy, ou direct VPS), googlevideo répond 403. D'où `AudioFormat.viaProxy` persisté et `boundProxy` dans `fetchGooglevideo`. Rotation de proxy *après* résolution = mort du titre, pas un retry utile.

Innertube signé OAuth (compte Google de l'utilisateur MHC, cookies persistés hors proxy public) reste en sortie directe VPS : on ne veut pas envoyer ces cookies à un open-proxy. Le chemin proxy gratuit sert yt-dlp extracteurs, le relais audio, Genius (allowlist), le canary Google 204. Mélange des deux = fuite de session.

4.2 Harvest : sources du pool

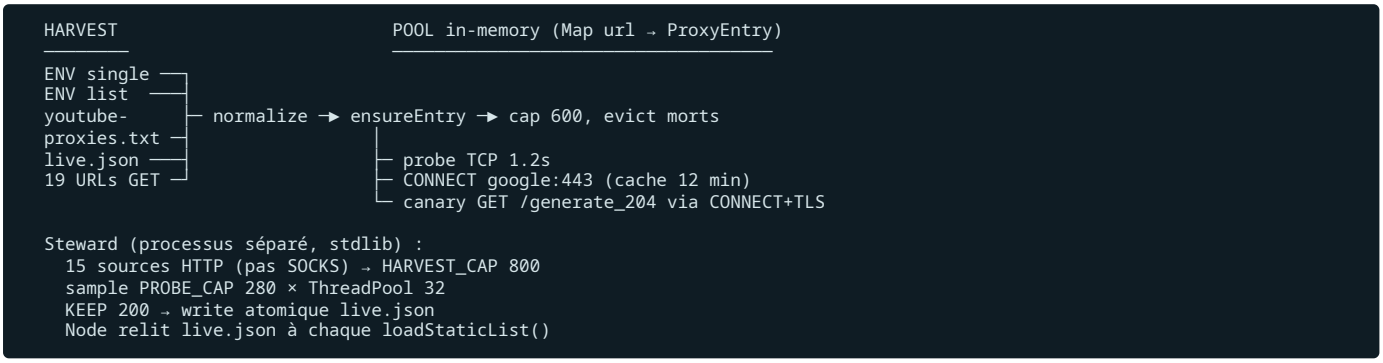
Module : `api/src/youtube/youtubeProxy.ts`. Ordre d'injection dans `loadStaticList()` puis, si free ON, `refreshFreeProxies()`.

1. `YOUTUBE_HTTP_PROXY` (ou à défaut `HTTPS_PROXY`) — un proxy fixe, auth autorisée (`allowAuth: true`). Utile pour un residential payant. **Pas** mis en tête des pools perso (évite de brûler une seule IP payante pour tous).
2. `YOUTUBE_HTTP_PROXY_LIST` — CSV / espaces / points-virgules, *ou* chemin de fichier si `existsSync`.
3. Fichier volume `data/youtube-proxies.txt` (cwd processus API, donc dans le volume MHC).
4. `data/proxy-steward/live.json` — sortie du steward, triée par `score` décroissant, flag `stewardLoaded`.
5. Si `YOUTUBE_HTTP_PROXY_FREE` n'est pas `0/false/no` : 19 listes HTTP publiques (ProxyScrape HTTP+SOCKS5, proxy-list.download, TheSpeedX, clarketm, monosans HTTP+SOCKS5, jetkai, ShiftyTR, roosterkid, hookzof, mmpx12, zevtyardt, proxifly, sunny9577, prxchk, rdavydov, MuRongPIG). TTL cache listes `LIST_TTL_MS = 8 min`. Shuffle Fisher-Yates puis cap `POOL_CAP = 600`.

`youtubeProxyFreeEnabled()` : défaut *true* si `APP_ENV` / `NODE_ENV` ∈ {production, prod, preprod, dev} — donc ON sur le VPS même sans variable. Opt-out explicite `YOUTUBE_HTTP_PROXY_FREE=0`.

Normalisation : ligne `host:port` → `http://host:port`. Commentaires `#` ignorés. Auth sur listes publiques **strippée** (appâts credential stuffing). Hosts bloqués (RFC1918, localhost, metadata, `*.internal` / `*.local`) rejetés. SOCKS est accepté dans le pool Node pour yt-dlp `--proxy`, mais `fetchUrlViaProxy` refuse SOCKS (`SOCKS fetch unsupported`) : le relais googlevideo Node n'est que HTTP CONNECT.

Refresh fond : `startYoutubeProxyBackgroundRefresh()` au listen API, intervalle `BG_REFRESH_MS = 5 min`, `unref()` pour ne pas retenir le process. Si pool utilisable < `LOW_POOL_REFRESH = 18`, force refresh. Après refresh : `canaryWarmSample()` sonde jusqu'à 24 HTTP sans gvHits, par lots de 6, canary `https://www.google.com/generate_204`.



4.3 ProxyEntry, constantes, ranking

Type interne (non exporté) :

Champ	Rôle
`url`	Clé normalisée `http(s)://host:port` (auth seulement si source fixe autorisée)
`fails`	Compteur d'échecs TCP/yt-dlp/CONNECT
`lastFailAt` / `lastOkAt`	Horodatages ms ; cooldown et ranking
`deadUntil`	Skip jusqu'à cette date (quarantaine)
`gvHits` / `gvMiss`	Score relais googlevideo 206/200 vs échec
`connectOkUntil`	CONNECT google:443 OK, cache ~12 min (évite re-sonder)

Constante	Valeur	Effet
`MAX_FAILS`	2	À 2 fails dans la fenêtre cooldown → `usable()=false` ; après cooldown, fails remis à 0
`COOLDOWN_MS`	4 minutes	Quarantaine et evict delay
`EVICT_AFTER_FAILS`	4	Sortie définitive du pool + drop lease ; refresh fond si free ON
`POOL_CAP`	600	Plafond Map ; éviction des moins scorés sans gvHits
`USER_POOL_SIZE`	24 (env `YOUTUBE_PROXY_USER_POOL` clamp 8–48)	Taille du lease exclusif par compte MHC
`LIST_TTL_MS`	8 min	Cache des 19 listes publiques
`BG_REFRESH_MS`	5 min	Timer process API
`LOW_POOL_REFRESH`	18	Seuil « pool trop maigre » → force
`PROBE_TIMEOUT_MS`	1 200 ms	TCP connect
CONNECT probe	2 400 ms	Un port 80 ouvert n'est pas un proxy
Canary	URL Google generate_204	204 ou 200 = atteint vraiment Google

`usable(e)` : pas dans `deadUntil` ; si fails ≥ 2 et encore dans les 4 min → non ; si fails ≥ 2 et cooldown écoulé → reset fails. `rankPick` : score `gvHits*20 + (connectOk?15:0) + lastOkAt/1e7 - fails*5 - gvMiss*8`, puis round-

robin dans le top (8 à 48, ~1/8 du lot). `evictDead` : drop si fails ≥ 4 et lastFail ancien de plus de 4 min ; si size > 600, drop la queue sauf gvHits > 0.

Maps de lease : `proxyLease: url → userId` (exclusivité), `userLeases: userId → Set<url>`. `dropFromPool` retire les deux. Hash : FNV-1a 32 bits `hash32(userId + "\0" + url)` pour un ranking déterministe par utilisateur (même user reprend les mêmes proxies « à gvHits égal »).

> Interprétation ops : `gvWinners` dans les stats = proxies encore usable avec au moins un gvHit. C'est le vrai trésor. Un poolSize 600 avec usable 12 et gvWinners 2 = on va 429. Le steward vise à remonter usable avant que Node ne brûle yt-dlp 12 s sur des morts.

4.4 Algorithme leasePoolForUser

Signature interne : `leasePoolForUser(userId: string): string[]`. Appelée depuis `nextYoutubeProxy` dès qu'un `userId` est fourni. Pas de lease si scan/warm oublient l'uid (régression à éviter : c'est le bug historique « un titre + N comptes = une IP »).

1. Créer le `Set` user s'il n'existe pas.
2. Drop des URLs mortes / plus dans le pool / plus usable ; si `proxyLease[url]===userId`, libérer.
3. Tant que `set.size < USER_POOL_SIZE` (24 par défaut) : candidats = entries usable, pas déjà dans le set, **sans owner ou owner === userId** (exclusif). Trier par `gvHits*1000 + (hash32(userId+"\0"+url) % 10000)` desc. Assigner `proxyLease.set(url, userId)` et `set.add`.
4. **Overflow partagé seulement si `set.size < 6`** (pénurie). On ajoute des usable non détenus, triés gvHits desc, jusqu'à 6. Ces overflow ne prennent *pas* le lease exclusif (pas de `proxyLease.set` dans cette boucle) : ils sont un appoint temporaire, pas un vol du voisin.

```
leasePoolForUser(uid)
    │
    ├─ prune set : !pool || !usable → delete + unlock
    │
    ├─ while |set| < 24:
    │     unused = usable ∧ not in set ∧ (no owner ∨ owner==uid)
    │     sort (gvHits*1000 + hash32(uid\|\url)%10000)
    │     lock url → uid   // exclusif
    │
    └─ if |set| < 6:                    // PÉNURIE seulement
          append shared usable by gvHits
          (pas de lock exclusif)

Conséquence : A et B ne se battent plus pour le même « meilleur » proxy
tant que le pool global a ≥ 24×N_users endpoints usable.
Si le steward ne nourrit plus, overflow<6 re-crée volontairement du partage
plutôt que de laisser un compte à 0 proxy (écoute morte).
```

Dimensionnement : 24 × nombre de comptes MHC actifs. Avec 10 users = 240 leases, compatible POOL_CAP 600 si usable tient. Si steward ne sort que 38 vivants (test du jour), 24×2 users déjà saturent : d'où overflow et l'importance du loop 5 min. Env `YOUTUBE_PROXY_USER_POOL=8` sur un VPS pauvre en vivants ; 48 seulement si harvest abondant (risque de monopoliser les gvWinners).

Sans `userId`, `nextYoutubeProxy` préfère les unleased (`preferUnleased`), puis relâche. Le proxy fixe ENV n'est injecté en tête des attempts que si `!userId`.

4.5 nextYoutubeProxy, attempts, stripe

`nextYoutubeProxy(exclude, userId?)` : `ensureYoutubeProxyPool()`, puis candidats = lease user filtrés par exclude, ou pool global unleased. Si vide : force refresh au plus toutes les 15 s, re-pick, puis pick y compris leased, sinon `null` (direct VPS — souvent LOGIN_REQUIRED, dernier recours).

`youtubeProxyAttempts({ max, includeDirect, directLast, shuffle, probe, userId })` : max clamp 1–16, défaut 4. Boucle jusqu'à `want = max(max*3, 16)` d'URLs brutes. Probe HTTP CONNECT par lots de 8 (défaut probe ON si free). Échec probe → fails++, deadUntil min(cooldown, 90 s). Direct (`null`) en tête ou en queue selon `directLast`. Live résolution : `directLast: true`, `includeDirect: false` sur la stripe pour ne pas brûler l'IP VPS en parallèle des 3 proxies.

`youtubeProxyStripe(userId, n=3)` : wrapper `youtubeProxyAttempts({ max:n, userId, includeDirect:false, probe:true, shuffle:false, directLast:true })`. Trois proxies *distincts du pool user*, déjà CONNECT-ok, pour la course.

Marquage : `markYoutubeProxyFailure(proxy, 'tcp'|'gv')` incrémente fails et gvMiss si gv ; à MAX_FAILS pose deadUntil 4 min ; à EVICT drop. `markYoutubeProxySuccess` reset fails, lastOkAt, gvHits++. `isProxyWorthRetry` regex large (LOGIN_REQUIRED, bot, 403, 429, 502–504, timeout, SOCKS, TLS, format unavailable…).

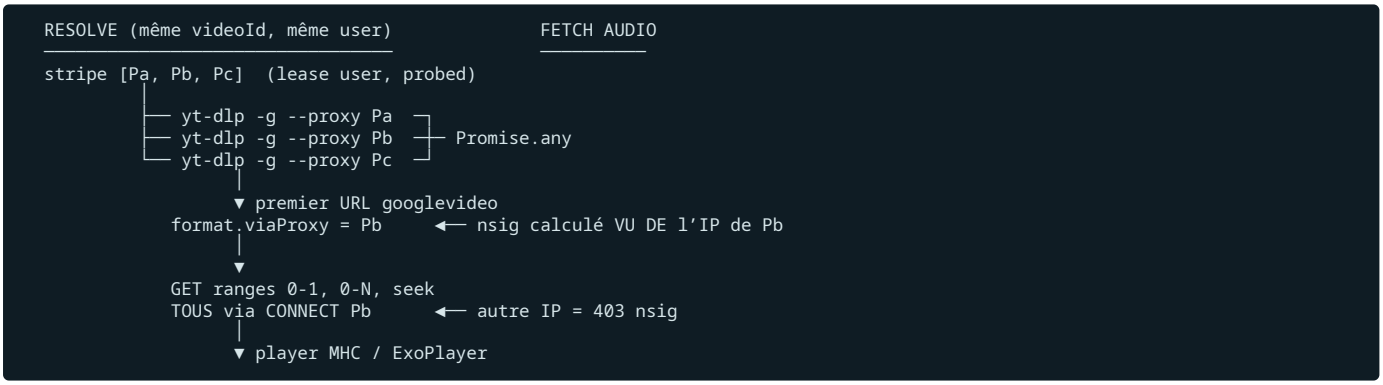
Sondes amont googlevideo : `probeGooglevideoUpstream` GET Range `bytes=0-1`, OK si 200/206, avant de piper le flux au client (évite de coller un 5xx au player Android).

4.6 Course Promise.any et boundProxy sticky

Dans `api/src/youtube/yt.ts`, chemin live `audioFormatViaYtDlpFast` (extrait) :

1. Si cooldown yt-dlp pour ce user → throw immédiat `yt-dlp cooling`.
2. `tryProxy(proxy)` : boucle extractor-args yt-dlp, `ytDlpGetUrl(-g)`, success → `markYoutubeProxySuccess` et `AudioFormat.viaProxy = proxy`.
3. Si `opts.live` : `stripe = youtubeProxyStripe(userId, 3)` puis `Promise.any(stripe.map(tryProxy))`. Le premier qui rend une URL gagne ; les autres courses sont abandonnées au niveau Promise (les proc yt-dlp déjà lancés restent bornés par le sémaphore gate, §4.8).
4. Si la course échoue : fallback séquentiel `youtubeProxyAttempts({ max:3, includeDirect:false, shuffle:true, probe:true, userId })`.

Le format gagnant porte `viaProxy`. Dans `api/src/media/stream.ts`, `fetchGooglevideo(url, range, { boundProxy: format.viaProxy, userId })` : si `boundProxy` HTTP, *un seul* essai via ce proxy (`tryOnce`), pas de rotation. SOCKS bound → pas de fetch Node (yt-dlp a déjà le fichier ou on échoue proprement). Ranges suivants (seek Android, prefetch 64 KiB vs tête) réutilisent la même IP.



> Ne jamais « retry 403 googlevideo avec le proxy suivant » sans re-résoudre `-g` sur ce proxy. Un 403 sur boundProxy = titre à re-décrypter, pas à rotator aveuglément. C'est la différence entre relais maison et CDN public.

Warm / disk : `enqueueStreamWarm(ids, userId)` propage l'uid jusqu'au même `viaProxy`. Health scan : `nextTrackId()` retourne `{ id, userId }` (MIN(user_id) du titre dans libraryUlikes) pour que le probe de santé n'emprunte pas le lease d'un autre compte au hasard — tout en ne vérifiant chaque `track_id` YouTube qu'une fois (table `track_health` globale).

4.7 fetchUrlViaProxy, allowlist, SSRF

`fetchUrlViaProxy(targetUrl, proxyUrl, { method, headers, timeoutMs })` : si proxy null → `fetch` direct. Si proxy non HTTP → reject SOCKS. Parse target ; `isAllowedProxyTarget(hostname)` : racines `google.com`, `googlevideo.com`, `youtube.com`, `youtu.be`, `ytimg.com`, `ggpht.com`, `genius.com` (exact ou suffixe `.domain`). Sinon reject `proxy target not allowed` — l'API ne devient pas un open-relay.

Côté proxy : `isBlockedProxyHost` — localhost, `::1`, `0.0.0.0`, metadata GCP, `*.internal` / `*.local`, IPv6 link-local/ULA grossier, IPv4 0/127/10/192.168/172.16-31/169.254. Même garde dans le steward Python (`ipaddress` private/loopback/link-local/reserved/multicast).

Implémentation : `http.request` method CONNECT, path `host:443`, timeout ≤ 5 s sur le tunnel. Status CONNECT ≠ 200 → fail. Puis `https.request` avec `createConnection` = `tls.connect({ socket, servername })` (SNI). Body streamé en `Readable.toWeb` vers `Response` Web. Timeouts TLS séparés. Canary et probe GV réutilisent cette primitive.

Genius est allowlisté pour les paroles via le même relais si besoin (module `lyricsGenius.ts` du 20 sept.) : pas d'ouverture vers le web entier.

4.8 ytDlpGate : user d'abord, global exceptionnel

Fichier `api/src/media/ytDlpGate.ts`. Sans gate, warm + stripe × formats ⇒ 30–40 proc yt-dlp, ~2 Go RAM, CPU 100 %, logs ERROR YouTube en boucle.

Paramètre	Défaut	Rôle
`YTDLP_MAX_CONCURRENT`	4 (clamp 1–12)	Sémaphore processus **global**
`YTDLP_LIVE_RESERVED`	1	Slot réservé écoute live ; warm cap = MAX−1
`YTDLP_BOT_COOLDOWN_MS`	300 000 (5 min, clamp 30 s–1 h)	Durée cooldown après bot/429
`GLOBAL_BOT_WINDOW_MS`	120 000	Fenêtre 2 min
`GLOBAL_BOT_USER_THRESHOLD`	3	Users distincts bot → cooldown global

`noteYtDlpFailure` ne s'arme que sur messages bot/login/rate-limit/unavailable typiques, et seulement si l'appelant n'a pas passé `noteFailure:false` (la boucle proxy décide souvent elle-même). Cooldown écrit dans `userCooldownUntil[userId]`. Tableau `recentBotHits` : si ≥ 3 userId distincts dans 2 min → `cooldownUntil` global (tout le monde, y compris anon). Un seul user qui 429 ne doit plus geler le player du voisin.

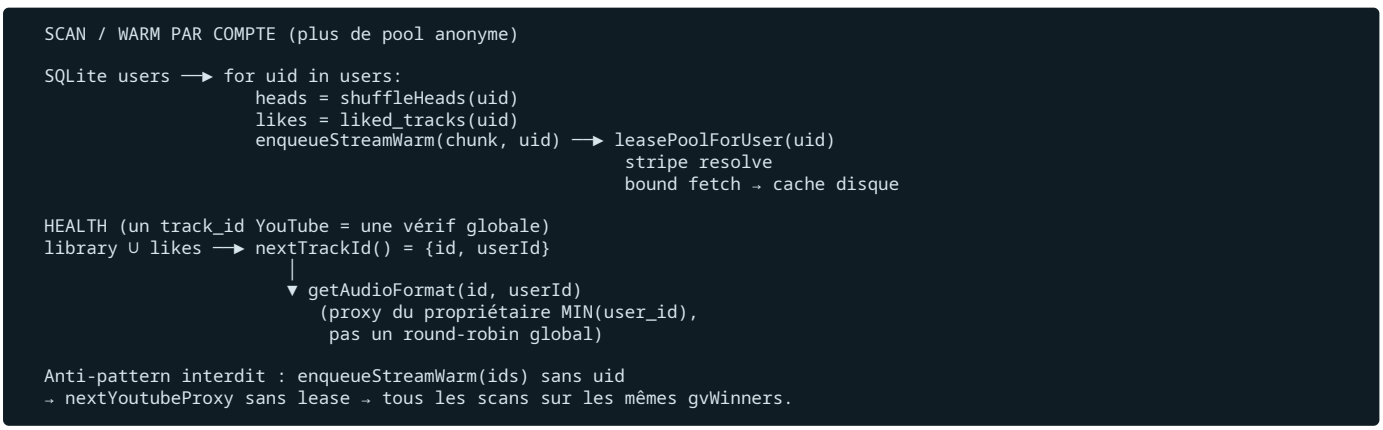
File d'attente : `liveWaiters` prioritaire sur `waiters` (warm). `bypassCooldown` existe pour tester un autre proxy malgré le cooldown user — à utiliser avec parcimonie (commentaire module : le cooldown ne doit pas bloquer la rotation, mais on ne l'arme qu'après épuisement des tentatives).

Interaction avec le lease : un user en cooldown 5 min laisse ses 24 proxies au chaud (gvHits inchangés). Les autres users continuent. Le sémaphore 4 reste le plafond CPU, même avec 10 leases.

4.9 Scan libraryHealth et libraryWarmSweep

`libraryHealth.ts` : détecte les YouTube IDs morts avant que le téléphone n'attende 40 s un remplaçant. Table `track_health(track_id, state, checked_at)` globale (ok/replaced/dead/pending). Tick défaut 5 s, start delay 120 s, idle stream 20 s avant travail lourd. Recheck ok 7 jours, retry dead 3 jours. SQL `nextTrackId` : UNION library_tracks ∪ liked_tracks, GROUP BY track_id, `MIN(user_id) AS userId`, priorité à un user différent du dernier (`lastHealthUserId`) pour round-robin comptes, likes d'abord, jamais-vérifiés d'abord, created_at DESC. Retour `{ id, userId }` — le probe `getAudioFormat` doit passer cet uid au proxy.

`libraryWarmSweep.ts` : tous les `users.id`, têtes shuffle (64) + recent (40) + likes (limite env 1500, clamp 50–5000). Chunks likes 16 + heads 8. Pour chaque chunk : `enqueueStreamWarm(chunk, uid)` **avec uid**, plus disk warm. Pause playback hot 90 s. Intervalle défaut 6 h, start delay 180 s, kill switch `LIBRARY_WARM_SWEEP=0`.



4.10 Steward Python

Chemin : `scripts/proxy-steward/steward.py`, README adjacent, `requirements.txt` vide (stdlib only). Classe `ProxyStewardAgent` : collect → probe → score → persist. CLI `--once` (défaut) ou `--loop` (sleep 300 s). UA `MHC-steward/1.0`.

15 sources HTTP (pas SOCKS, contrairement au Node qui avale aussi socks5 pour yt-dlp). Harvest parallèle `ThreadPoolExecutor(max_workers=len(SOURCES))`, timeout 8 s, cap 800 uniques. Probe : sample 280, 32 workers, timeout 2.5 s. Pour chaque candidat : TCP, `CONNECT www.google.com:443`, exiger 200, wrap SSL, `GET /generate_204`, exiger 204 ou 200. Score heuristique `0.55 + 0.45 / (1+latency)`, keep 200, tri desc. Persist : mkdir `data/proxy-steward`, mkstemp + `os.replace` → `live.json` `{"updated": epoch_ms, "proxies": [{"url","score","latency"}]}`.

Test réel du 20 septembre 2026 : `python3 steward.py --once` → **38 vivants en 28 s**. C'est peu vs POOL_CAP 600, mais 38 CONNECT+204 valides valent mieux que 600 lignes mortes que yt-dlp paierait 12 s chacune. Le loop 5 min recrée le fichier ; Node relit via `loadStaticList` à chaque ensure. Steward ne sonde *jamais* une destination arbitraire (pas de SSRF ops).

> Déployer le steward *dans* le volume ou un sidecar qui écrit le même path `data/proxy-steward/live.json` vu par l'API. Un cron hôte qui écrit ailleurs est inutile. `--loop` dans un container `restart: unless-stopped` est le mode prod visé ; `--once` pour debug.

Différence Node vs Python : Node mélange SOCKS (yt-dlp) + HTTP (relais) + 19 listes ; Python ne garde que HTTP CONNECT-capable, pré-chauffe les 200 meilleurs pour que `stewardLoaded=true` et que le lease ait de la matière dès le boot, avant le premier refresh 8 min des listes Node.

4.11 Fichiers, stats, hors-périmètre, live

Fichier	Rôle le 20 sept.
`api/src/youtube/youtubeProxy.ts`	Pool, lease, stripe, fetch CONNECT, stats, SSRF
`api/src/youtube/yt.ts`	yt-dlp `-g`, Promise.any stripe live, `viaProxy` sur AudioFormat
`api/src/media/ytDlpGate.ts`	Sémaphore 4, cooldown user/global
`api/src/media/stream.ts`	`fetchGooglevideo` boundProxy, `enqueueStreamWarm(ids, uid)`
`api/src/media/libraryHealth.ts`	`nextTrackId() → {id,userId}`
`api/src/media/libraryWarmSweep.ts`	Sweep multi-users avec uid
`scripts/proxy-steward/steward.py`	Guardian stdlib, live.json atomique

`youtubeProxyStats()` expose : `enabled`, `poolSize`, `usable`, `freeEnabled`, `gvWinners`, `users` (taille `userLeases`), `leased` (taille `proxyLease`), `stewardLoaded`. `userProxyPoolStats(userId)` : size/usable du set perso. Brancher ces champs sur un endpoint admin existant (ne pas inventer de secrets dans les logs : pas d'URL proxy complètes en info, hosts seulement si debug).

Ce que l'on ne fait pas

- Envoyer les cookies OAuth Innertube utilisateur dans un proxy public / liste gratuite. Innertube signé reste en direct VPS.
- Implémenter un fetch SOCKS côté Node (yt-dlp `--proxy socks5://` uniquement).
- Ouvrir CONNECT vers des hôtes hors allowlist (anti open-relay).
- Utiliser des listes publiques `user:pass` (strippées ; probablement des honeypots).
- Cooldown global au premier 429 d'un seul compte.
- Retry googlevideo 403 en changeant d'IP sans nouveau `-g`.

Live MHC 20 sept. 2026

Élément	Valeur
URL	`https://music.hubera.cloud` (dual `plm.delhomme.ovh` / ytmusic NPM → `ytmusic:8787`)
appVersion API	`p+1.3.265`
Image overlay	`ytmusic:1.3.265-lib`
Volume	`ytmusic_ytmusic_data` (immuable)
Git	`117773c` sur `fix/stream-live-warm-247` → PavelDelhomme/YTMusic
applicationId	`ovh.delhomme.ytmusic`

Pas de pull GHCR Music « pour aligner » : l'overlay local est la source de vérité runtime. Rebuild image sur le VPS depuis le tree déployé.

5. Bibliothèque web MHC 1.3.265

Avant : `LibraryPage` faisait `songs.map` vers N `TrackRow` montés d'un coup. Coût : layout/paint proportionnel à la taille de bibliothèque (milliers de nœuds DOM), couverture `warmHead` ~1.6 Mo par lignes visibles *et* hors écran (prefetch audio systématique), `refresh()` bibliothèque à chaque navigation de route (store sans TTL), `isLiked(id)` en O(n) sur le tableau `liked` à chaque ligne → O(n^2) au rendu de la liste.

Après (commit `117773c`) :

- `VirtualTrackList` (`web/src/components/media/VirtualTrackList.tsx`) : fenêtre scroll, `rowHeight` 72 px, overscan 12, spacers haut/bas. Seules les lignes visibles + overscan montent des `TrackRow`.
- `prefetch={false}` sur ces lignes : plus de warmHead 1.6 Mo × N. Le player / `streamPrefetch` gère le voisinage de lecture, pas la liste complète.
- API `GET /api/library?light=1&limit=10..40` : première peinture (mobile) via `getLibraryLight` ; le full `getFullLibrary` suit. Repair meta et taste-warm en fond, non bloquants.
- Store Zustand `LIB_TTL_MS = 10 * 60 * 1000` : pas de refetch full si `lastFullAt` < 10 min, sauf `refresh(true)`.
- `likedIds` / `libraryIds` en `Set` : `isLiked` / `isInLibrary` en O(1).

Pages utilisatrices : onglets Titres, J'aime, Historique, contenus de playlist — toujours `VirtualTrackList`. L'onglet Ajouts peut rester sur un extrait 30 titres pour la grille. Squelette `LibraryListSkeleton` pendant `partial`.

> Couplage proxy : une biblio qui ne précharge plus N titres au scroll réduit la pression yt-dlp et donc les 429. La virtualisation n'est pas cosmétique ; c'est un levier anti-bot au même titre que le lease.

6. Hubera Fuel 1.4.142

Commit `59005f6` branche `prod`, remote `PavelDelhomme/HuberaFuel`. `app.json` version `1.4.142`. Package Android `com.gasoiltracking.app` **inchangé**.

6.1 Persistance jauge

Table SQLite `fuel_gauge_readings` : `id`, `vehicle_id`, `recorded_at`, `liters`, `tank_capacity`, `source`, `trip_id`, `fill_up_id`, `odometer`. Index `(vehicle_id, recorded_at)`. Sources (`FuelGaugeSource` + labels FR) :

source	Libellé UI	Quand
`trip_start`	Départ trajet	Début de course / trajet libre
`trip_end`	Arrivée trajet	Fin de trajet
`fill_up`	Plein	Saisie station
`manual`	Saisie	Correction utilisateur
`model_burn`	Conso estimée	Modèle de brûlage entre points
`recompute`	Recalcul	Rebuild historique

Dédup (`fuelGaugeHistoryCore.ts`) : même source, même trip, $\Delta t \leq 20$ s, Δlitres < 0.05 → skip. Sync snapshot : `SNAPSHOT_SCHEMA = 2` inclut `gaugeReadings` dans l'export/import (hash de contenu ignore `exportedAt` pour stabilité).

6.2 UI

`FuelGaugeTimeline` : véhicule (`vehicle/[id]`, jusqu'à 80 points) et trajet (`trip/[id]`, « Jauge départ → arrivée »). Pastille / chip historique sur l'onglet trajets (filtre dates + véhicule). Slider jauge existant inchangé dans le principe ; l'historique le rend auditable (litres, %, source, horodatage relatif FR).

6.3 Live vs APK

Health web / image : **1.4.142**. Endpoint `/api/version` encore **1.4.138** = dernière APK enregistrée en base (canal OTA). L'overlay 1.4.142 n'est pas encore le binaire « officiel » servi aux téléphones. Conséquence : un Nothing/Blackview/Samsung en 1.4.141/138 ne verra pas la timeline tant que `adb install -r` 1.4.142 n'est pas fait. Ne pas forcer-update OTA sans overlay fumé.

> CORS / QR / `fuel.hubera.cloud` vs `gasoil-tracking.hubera.cloud` : dual-alias NPM historique. Ne pas supprimer l'hôte delhomme.ovh. Volume `gasoil_api_data` intact.

7. HuberaPress

Produit CMS : domaine `cms.hubera.cloud`, identité `cloud.hubera.cms`, stack Portainer `hubera-cms`, runtime Django+Next à côté (`hubera-cms-runtime`). Git local `HuberaWordPress`, GitHub `PavelDelhomme/HuberaPress` @ `aa4e07a` `main`.

Séparation des consoles (commit VTCBuilder `306b712` + Press) :

- `/admin/` — Next.js custom HuberaPress (tenants, pages, médias, blocs). Chrome (titres layout, sidebar `AdminSidebar`) dit **HuberaPress**, plus VTCBuilder.
- `/django-admin/` — Django staff (auth Django, pas la console métier chauffeur).

API publique `urls_public.py` : message « API HuberaPress — console /admin, Django staff /django-admin ». Login layout : titre « HuberaPress — Connexion Hubera ». Compte staff utilisé pour la vérif : `paul@delhomme.ovh` (mot de passe non documenté ici, uniquement coffre / Pass / hors Git).

VTCBuilder **consommera** ce CMS ; les Git restent séparés. Ne pas fusionner les stacks. DNS `vtcbuilder.com` encore ouvert (voir §12). Pages publiques CMS ≠ app chauffeur.

> Ne pas coller le mot de passe CMS dans un ticket, un PDF, un canal Slack, un commentaire Portainer. Rotation si le secret a fuité dans un chat précédent.

8. Hubera Jobs — titres live

Commit `6cc8ba15` branche `dev`, remote `PavelDelhomme/HuberaJobs`. 12 fichiers : `layout.tsx`, pages login / register / maintenance / home, `pageTitles.ts`, i18n `fr.ts`, Dockerfiles et compose (labels image). Objectif : le document.title, les H1 publics et les libellés compose disent **Hubera Jobs**, plus « JobbingTrack » sur le chemin utilisateur.

Reste : `frontend/public/manifest.json` et plusieurs `package.json` de services internes portent encore JobbingTrack (description packages, Playwright). Non bloquant pour le titre onglet live après rebuild frontend. Stack NPM : `jobs.hubera.cloud` → `jobbingtrack-prod-frontend:3000`, API `api.jobs.hubera.cloud` (record nested hors wildcard — vérifier DNS si 404). applicationId mobile `com.example.jobbingtrack_mobile` inchangé. Rebuild image frontend requis pour que le live affiche les nouveaux titres (sinon cache ancienne image).

9. apps/ versus products/

Meta-repo Hubera :

```
Hubera/
├── apps/www          # vitrine holding hubera.cloud uniquement
├── apps/admin        # outillage holding (pas un produit utilisateur)
├── platform/         # @hubera/app-contract, tokens, …
├── products/<slug>   # symlinks → clones Perso (Music, Fuel, Jobs, …)
├── kits/             # AllInOne, backoffice-lab
└── catalog.json      # source de vérité noms / domaines / stacks
```

`apps/www` = holding (catalogue, legal, updates.json). Ce n'est pas MHC, pas Fuel. Un lien erroné `products/www` a été corrigé dans `64a9876` pour pointer vers la vitrine sans en faire une « app utilisateur ». `products/` = applications que les gens ouvrent (music, fuel, jobs, cms, id, mail, ...). Cloudity = archive, listée dans `catalog.archive`, plus un produit.

Règle de contribution : code métier dans le Git du produit ; meta (catalogue, NPM hosts, règles zéro perte) dans Hubera. Pas de nested Cloudity dans le workspace produits.

10. Déploiements production du 20 septembre

Stack	Action réelle	Interdit / non fait
Music / ytmusic	Overlay image `ytmusic:1.3.265-lib`, volume conservé, appVersion `p+1.3.265`	Pas de `down -v`, pas de pull GHCR Music, pas de merge GitHub prod
Fuel	Code 1.4.142 poussé `prod` ; health image alignée ; `/api/version` reste 1.4.138	Pas d'overlay APK universel encore ; pas de wipe SQLite
Jobs	Push `dev` titres ; rebuild frontend à confirmer sur le live	Pas de rename container NPM
HuberaPress / VTC	Chemins /admin et /django-admin, chrome relabel	Pas de merge Git VTC ↔ Press
Hubera meta	catalog + README apps vs products	—

Procédure overlay Music type : build image tag `1.3.265-lib` sur le VPS depuis le tree, retag service Portainer, recreate container *sans* `-v`. Vérifier `GET /api/version` ou health : `p+1.3.265`. Steward : s'assurer que `data/proxy-steward/live.json` est dans le volume monté.

Procédure Fuel APK (à faire, §12) : EAS/local release 1.4.142 même keystore, `adb install -r` sur Nothing + Blackview + Samsung, puis seulement mettre à jour la version en base pour que `/api/version` suive.

11. Exploitation quotidienne

11.1 Steward

```
cd /chemin/YTMusic/scripts/proxy-steward   # dans l'arbre déployé / volume
python3 steward.py --once                  # debug, ~30 s, log live=N
python3 steward.py --loop                  # prod, toutes les 5 min
```

Logs attendus : `harvested n=…`, `probed live=…`, `wrote n=… live.json`. Si live=0 : listes publiques down ou filtrage réseau VPS (egress 80/443). Ne pas baisser les timeouts à l'aveugle ; 2.5 s × 280 × 32 workers est déjà agressif.

11.2 Variables d'environnement MHC (sans secrets)

Variable	Rôle
`YOUTUBE_HTTP_PROXY`	Proxy fixe (residential) ; auth OK ; pas en tête des pools perso
`YOUTUBE_HTTP_PROXY_LIST`	Liste ou fichier
`YOUTUBE_HTTP_PROXY_FREE`	Défaut ON en prod/dev ; `0` pour couper les 19 listes
`YOUTUBE_PROXY_USER_POOL`	8–48, défaut 24
`YTDLP_MAX_CONCURRENT`	Défaut 4
`YTDLP_BOT_COOLDOWN_MS`	Défaut 300000
`LIBRARY_WARM_SWEEP`	`0` coupe le sweep
`LIBRARY_HEALTH_TICK_MS`	Cadence scan titres

11.3 Si 429 / LOGIN_REQUIRED

1. Lire stats : `usable`, `gvWinners`, `users`, `leased`, `stewardLoaded`. Si stewardLoaded false → lancer steward, vérifier le mount.
2. Un seul user touché : attendre cooldown user 5 min, vérifier que son lease n'est pas 6 overflow partagés (pénurie). Réduire warm pour ce compte n'existe pas encore : baisser sweep global ou ticker health.
3. Trois users+ en 2 min : cooldown *global* 5 min — c'est normal, YouTube a classé le VPS. Ne pas scaler yt-dlp à 12 : ça aggrave. Steward + attendre.
4. Ne pas coller un cookie Google perso dans un proxy gratuit « pour débloquer ».
5. Ne pas `down -v` pour « reset le pool » : le pool est mémoire process ; recreate container sans -v suffit. Le volume garde live.json.
6. 403 googlevideo isolé : boundProxy / nsig, re-résoudre le titre, pas rotator l'IP du GET.
7. CPU 100 % yt-dlp : stripe 3 × warm trop large ; gate max 4 déjà là — couper `LIBRARY_WARM_SWEEP` temporairement.

11.4 Git au quotidien

Pousser Music sur `fix/stream-live-warm-247` jusqu'à décision de merge. Fuel sur `prod`. Jobs sur `dev` jusqu'à promotion. Press `main`. Origin URLs : HuberaFuel / HuberaJobs / HuberaPress (pas les anciens noms). Dossiers locaux inchangés.

12. Limites et suite

- **APK Fuel overlay 1.4.142** : code et health image prêts ; `/api/version` 1.4.138 jusqu'à overlay téléphones + update base. Schema snapshot 2 : anciens clients 1.4.138 ignorent les jauges ou doivent rester rétrocompatibles à l'import (tester un sync mixte avant OTA).
- **Music Git** : live = branche `fix/stream-live-warm-247` @ `117773c`, pas mergée dans une `prod` GitHub. Risque : un rebuild Portainer « depuis main » reviendrait en arrière. Documenter le git_ref Portainer.
- **Steward 38 vivants** : fragile si YouTube brûle plus vite que 5 min. Piste : residential `YOUTUBE_HTTP_PROXY` payant en appoint, toujours hors OAuth. Piste : sidecar steward dans le compose MHC.
- **vtcbuilder.com DNS** : encore à poser / dual-alias vers le runtime CMS ou l'app chauffeur selon décision produit. Ne pas pointer l'apex chauffeur sur /admin CMS.
- **Jobs manifest** : PWA / package.json encore JobbingTrack. Titres onglet OK après rebuild.
- **SOCKS Node** : toujours absent ; gvHits uniquement via HTTP CONNECT. Un gagnant SOCKS yt-dlp ne servira pas le relais Range Node.
- **Innertube OAuth direct** : l'IP VPS reste exposée sur ce chemin. Si botcheck Innertube, ce n'est pas le steward qui le sauve.
- **Cloudity** : rester archive. Pas de réintroduction de nested modules.

13. Annexe A — versions et URLs live

Produit	URL principale	Version / état 20 sept.
Holding www	https://hubera.cloud	vitrine `hubera-www`
Music	https://music.hubera.cloud	API `p+1.3.265`, image `ytmusic:1.3.265-lib`
Music historique	https://plm.delhomme.ovh	même backend `ytmusic:8787`
Fuel	https://gasoil-tracking.hubera.cloud	health 1.4.142 ; `/api/version` 1.4.138
Jobs	https://jobs.hubera.cloud	titres Hubera Jobs (après rebuild image)
Jobs historique	https://jobbingtrack.com	même frontend prod
CMS	https://cms.hubera.cloud	HuberaPress /admin et /django-admin
Updates Fuel/Music	https://hubera.cloud/updates/…	canaux OTA ; ne pas forcer 1.4.142 tant qu'APK non overlay

13.B applicationId et Git

Produit	applicationId	Dossier	GitHub	SHA 20 sept.
Music	`ovh.delhomme.ytmusic`	YTMusic	PavelDelhomme/YTMusic	117773c
Fuel	`com.gasoiltracking.app`	GasoilTracking	PavelDelhomme/HuberaFuel	59005f6
Jobs mobile	`com.example.jobbingtrack_mobile`	JobbingTrack	PavelDelhomme/HuberaJobs	6cc8ba15
Press	n/a (web)	HuberaWordPress	PavelDelhomme/HuberaPress	aa4e07a
VTCBuilder	n/a ce jour	VTCBuilder	PavelDelhomme/VTCBuilder	306b712
Meta	n/a	Hubera	PavelDelhomme/Hubera	64a9876

Volumes Docker immuables : `ytmusic_ytmusic_data`, `gasoil_api_data`. Réseau : `shared-network-copy`. NPM : créer un host Hubera, ne pas éditer l'historique.

13.C Glossaire

Terme	Définition opérationnelle
MHC	Hubera Music / YTMusic / « Pue La Merde » historique ; API Node + yt-dlp + web Vite + APK `ovh.delhomme.ytmusic`
GV / googlevideo	CDN audio/vidéo Google ; URLs signées (nsig) ; 206 Partial Content pour Range
Innertube	API interne YouTube/YTM ; chemin OAuth user = cookies, **direct VPS**, pas proxy gratuit
yt-dlp `-g`	Get-url : résout l'URL GV sans télécharger le média
boundProxy	Proxy HTTP avec lequel le nsig a été obtenu ; tous les GET Range du titre doivent l'utiliser
lease	Assignation exclusive url→userId jusqu'à mort/evict ; overflow seulement si pool perso < 6
stripe	3 proxies du lease user, course Promise.any sur le même videoId
steward	Processus Python stdlib qui pré-filtre CONNECT+204 et écrit live.json
overlay `-r`	Réinstall APK sans changer applicationId ni signature
holding	Hubera meta + www ; pas Cloudity
nsig	Paramètre de l'URL GV lié à l'IP/player ; IP différente ⇒ 403

13.D Constantes proxy (mémo opérateur)

```
MAX_FAILS=2     COOLDOWN=4min     EVICT=4     POOL_CAP=600
USER_POOL=24 (YOUTUBE_PROXY_USER_POOL 8..48)
refresh Node=5min     TTL listes=8min     LOW_POOL=18
yt-dlp max=4     live reserved=1     bot cooldown user=5min
global bot si 3 users / 2 min
steward KEEP=200  PROBE_CAP=280  WORKERS=32  LOOP=5min
test 20/09/2026 : 38 live / 28 s
allowlist CONNECT : google googlevideo youtube youtu.be ytimg ggpht genius
block : RFC1918 localhost metadata *.internal *.local
```

Fin du récapitulatif technique Hubera — 20 septembre 2026 — Pavel Delhomme. Aucun secret. Aucun mot de passe CMS. Document destiné à l'exploitation et à la revue de code, pas au marketing.